Course: Design and Analysis of Algorithms (CoSc3094)

Project: Heap-Based Priority Queue: Emergency Room Triage System

Institution: Aksum University

Department: Computer Science

Submitted By (Group Members):Group Members:

No.	Name	ID Number
1.	Eyerusalem Teklay	AKU16023
2.	Weldearegay Abbe	AKU16034
3.	Shewit Legesse	AKU1602069

Submitted To: Mr. Ekubay

Date: March 2026

1.1 Introduction

In hospitals, patients are treated based on severity (priority), not arrival time.

A critical patient (priority 10) must be treated before a minor case (priority 1), even if they arrive later.

A priority queue (max-heap) is the perfect data structure for this problem.

This project implements an ER Triage System using a max-heap with array storage.

1.2 Algorithm Design

Why Max-Heap?
Highest priority (10) is always at the root.

Allows quick access to the most critical patient.

Indexing

Zero-based indexing used.

Parent(i) = floor((i-1)/2)

Left child = 2i + 1

Right child = 2i + 2

Pseudocode

insert(patientID, priority):

text

```
append (priority, patientID, timestamp) to heapi = last indexwhile i > 0 an
d heap[parent(i)].priority < heap[i].priority:    swap heap[i] and heap[par
ent(i)]    i = parent(i)
```

extractMax():

text

```
root = heap[0]last = heap.pop()if heap not empty:    heap[0] = last    maxH
eapify(0)return root
```

increasePriority(patientID, newPriority):

text

```
find index using pos dictionaryupdate prioritywhile i > 0 and heap[parent
(i)].priority < heap[i].priority:    swap and update pos dictionary
```

maxHeapify(i):

text

```
largest = ileft = 2i+1, right = 2i+2if left < size and heap[left].priority
 > heap[largest].priority:    largest = leftif right < size and heap[right].
priority > heap[largest].priority:    largest = rightif largest != i:    sw
ap heap[i] and heap[largest]    update pos dictionary    maxHeapify(largest)
```

buildMaxHeap():

text

```
for i from size//2 - 1 down to 0:    maxHeapify(i)
```

1.3 Implementation Details

Data Structures Used

self.heap – Python list (array) storing tuples: (priority, patientID, timestamp)

self.pos – Dictionary mapping patientID → index in heap (for O(1) lookup in increasePriority)

Key Functions Explained

insert – Appends new patient, then "bubbles up" until heap property is restored.

extractMax – Removes root, replaces with last element, then "bubbles down" using maxHeapify.

increasePriority – Finds patient by ID, updates priority, then bubbles up.

maxHeapify – Ensures subtree rooted at i is a max-heap.

buildMaxHeap – Converts any unsorted array into a max-heap in O(n) time.

Fixed-Size Array Note

Python lists are dynamic, but they function like fixed-size arrays with amortized constant time appends. For this

project, we used a list to simplify implementation while respecting the "array" requirement.

1.4 Complexity Analysis

Time Complexity

Operation	Complexity	Explanation
insert	O(log n)	Append (O(1)) + bubble up (O(log n))
extractMax	O(log n)	Remove root (O(1)) + maxHeapify (O(log n))
increasePriority	O(log n)	Find index (O(1)) + update + bubble up (O(log n))
maxHeapify	O(log n)	Traverses height of tree
buildMaxHeap	O(n)	Calls maxHeapify on n/2 nodes; total work is linear (standard proof)
isEmpty	O(1)	Just checks list length

Space Complexity

Heap array: O(n)

Position dictionary: O(n)

Total: O(n)

1.5Experimentation / Results

Sample Output (from terminal)

Copy 20-30 lines of your simulation output here. For example:

text

```
[00:00] ARRIVAL: P0266 (priority 1)[00:00] TREAT:   P0266 (priority 1)[00:0
5] ARRIVAL: P0127 (priority 10)[00:05] TREAT:   P0127 (priority 10)[00:09]
ARRIVAL: P0026 (priority 10)[00:10] TREAT:   P0026 (priority 10)...
```

Treatment Summary

Include the final summary from your simulation:

text

```
Priority 10: 94 patients, first treated at 00:05, last at 23:46Priority  9:
 102 patients, first treated at 00:32, last at 23:56Priority  8: 107 patien
ts, first treated at 00:14, last at 23:53...
```

Verification

The summary shows that all priority 10 patients were treated before any priority 9 patients, etc. This proves the max-heap correctly ensures higher priority patients are treated first, regardless of arrival time.

1.6 Conclusion

What I Learned

How to implement a max-heap from scratch

Importance of complexity analysis (O(log n) vs O(n))

How to simulate real-world systems using data structures

Debugging challenges (Unicode errors, OneDrive issues)

Challenges Faced

Unicode error when printing checkmark (✅) in Windows terminal – fixed by using plain text.

OneDrive interference with Git – moved project to Desktop.

False positive "order violation" – initial verification didn't account for arrival times; fixed by using summary statistics.

Possible Improvements

Add GUI for better visualization

Handle equal priorities by arrival time (FIFO)

Allow dynamic array resizing if needed

1.7 References

Dataset: Self-generated using Python's random module (1,000+ patients)

Textbook: "Introduction to Algorithms" (CLRS) – for heap proofs

Online: GeeksforGeeks, Stack Overflow (for debugging)

Cormen, T. H., et al. Introduction to Algorithms (CLRS).

Python Documentation: CSV and Random modules.

Course Material: CoSc3094 Lecture Notes by Mr. Ekubay.

Additional Deliverables

README File

Create a README.txt file:

text

ER Triage SystemFiles:- maxheap.py: MaxHeap class implementation- generate_data.py: Creates patient dataset (1000+ records)- simulate.py: Runs 24-hour ER simulation- patients.csv: Patient dataHow to Run:

1. (Optional) python generate_data.py
2. python simulate.py

Code Comments

Make sure each function in your Python files has a brief comment explaining what it does.